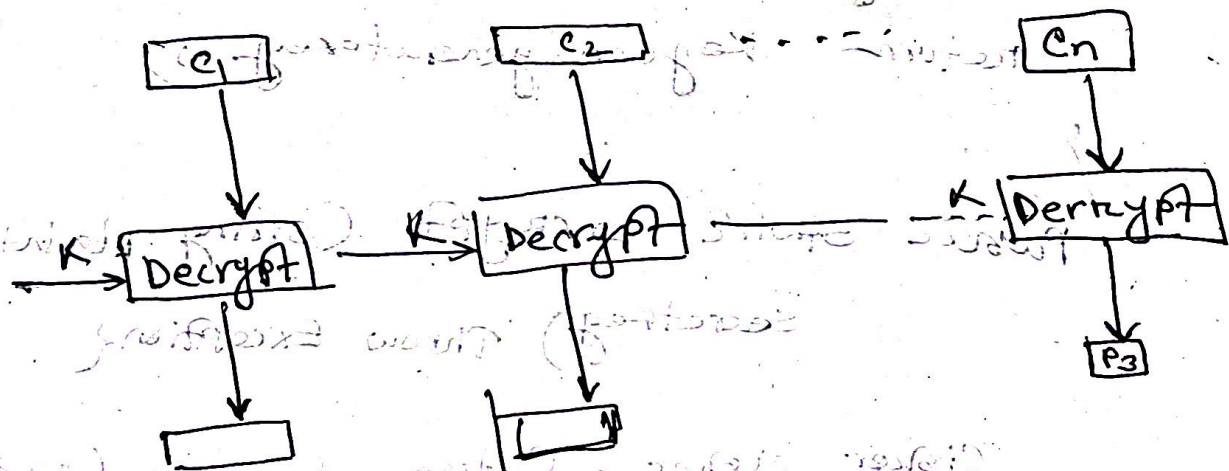
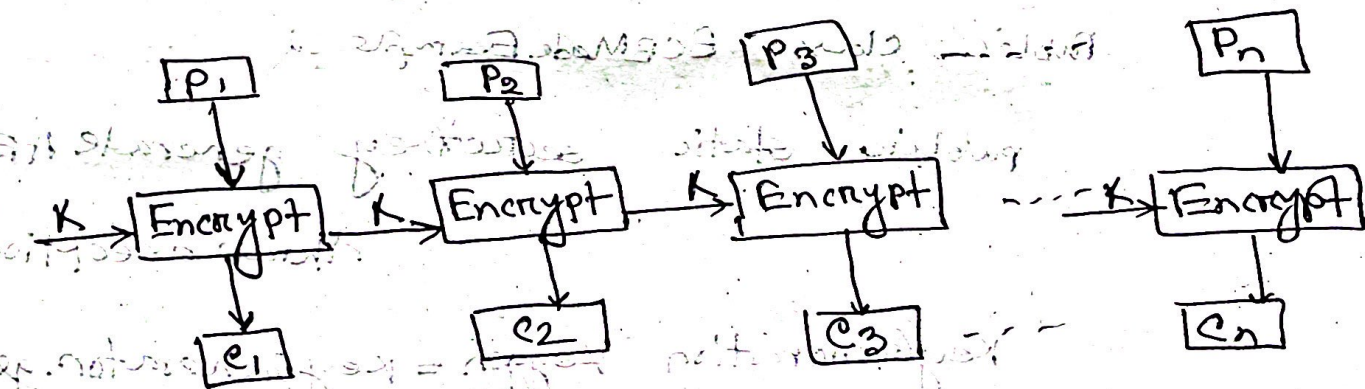


Md. Joyhel Abedin Joy

IT-21058

ECB (Electric codebook) : In an electric codebook each block of bits of plain text is encoded independently with the same key.

Block Diagram:



IT-21058

Java implementation of ECB

```
import javax.crypto.Cipher;  
import javax.crypto.KeyGenerator;  
import javax.crypto.SecretKey;  
import javax.crypto.spec.SecretKeySpec;  
import java.util.Base64;
```

Public class ECBModeExample2

```
public static SecretKey generateAESKey()  
    throws Exception
```

```
{  
    KeyGenerator keyGen = KeyGenerator.getInstance("AES");
```

```
    keyGen.init(128);
```

```
    return keyGen.generateKey();  
}
```

```
public static String encrypt(String plaintext,  
    SecretKey secretKey) throws Exception
```

```
{  
    Cipher cipher = Cipher.getInstance("AES/ECB/  
        PKCS5Padding");
```

```
Cipher.init(Cipher.ENCRYPT_MODE, secretKey);
```

```
byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());
```

```
return Base64.getEncoder().encodeToString(encryptedBytes);
```

```
}
```

Example 11: Decrypt cipher text using AES in ECB mode

```
public static String decrypt(String encryptedText,
                              SecretKey secretKey) throws Exception {
```

```
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5");
```

```
    cipher.init(Cipher.DECRYPT_MODE, secretKey);
```

```
    byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(encryptedText));
```

```
    return new String(decryptedBytes);
```

```
public static void main(String[] args) {
```

```
    try {
        secretKey = generateAESKey();
```


IT-21058

```
String original = "Hello cyber world";
```

```
String encrypted = encrypt(original, key);
```

```
String decrypted = decrypt(encrypted, key);
```

```
System.out.println("Original Text: " + original);
```

```
System.out.println("Encrypted text: " + encrypted);
```

```
System.out.println("Decrypted text: " + decrypted);
```

```
catch (Exception e) {
```

```
    e.printStackTrace();
```

output:

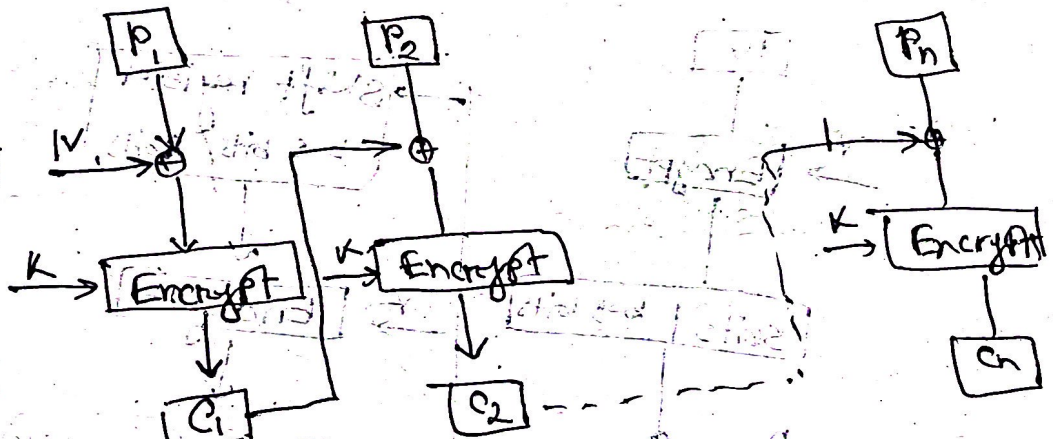
original Text : Hello cyber world !

Encrypted text: fRQ+*yKBP 3ANx IV4IQWPIQRWRgv

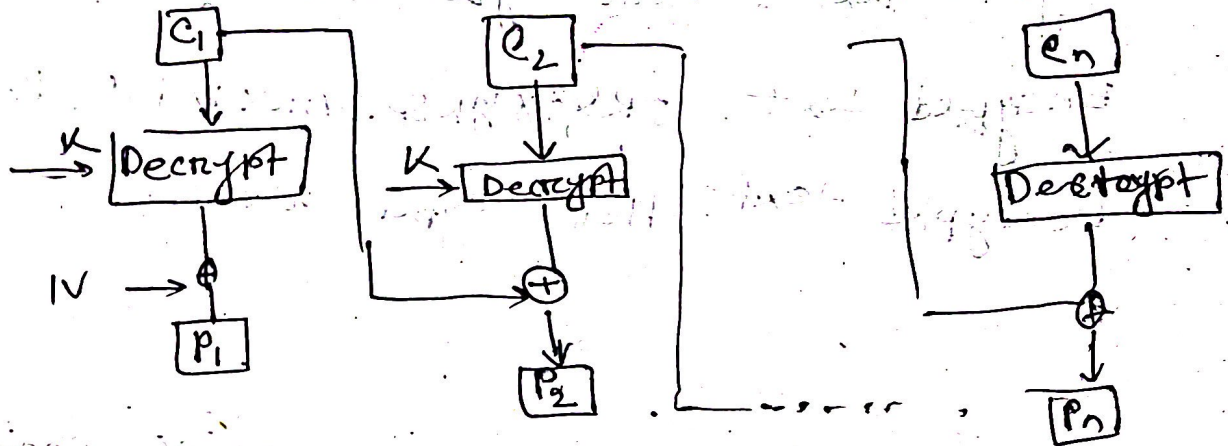
Decrypted text: Hello cyber world !

CBC (cipher Block chaining)

Encryption :

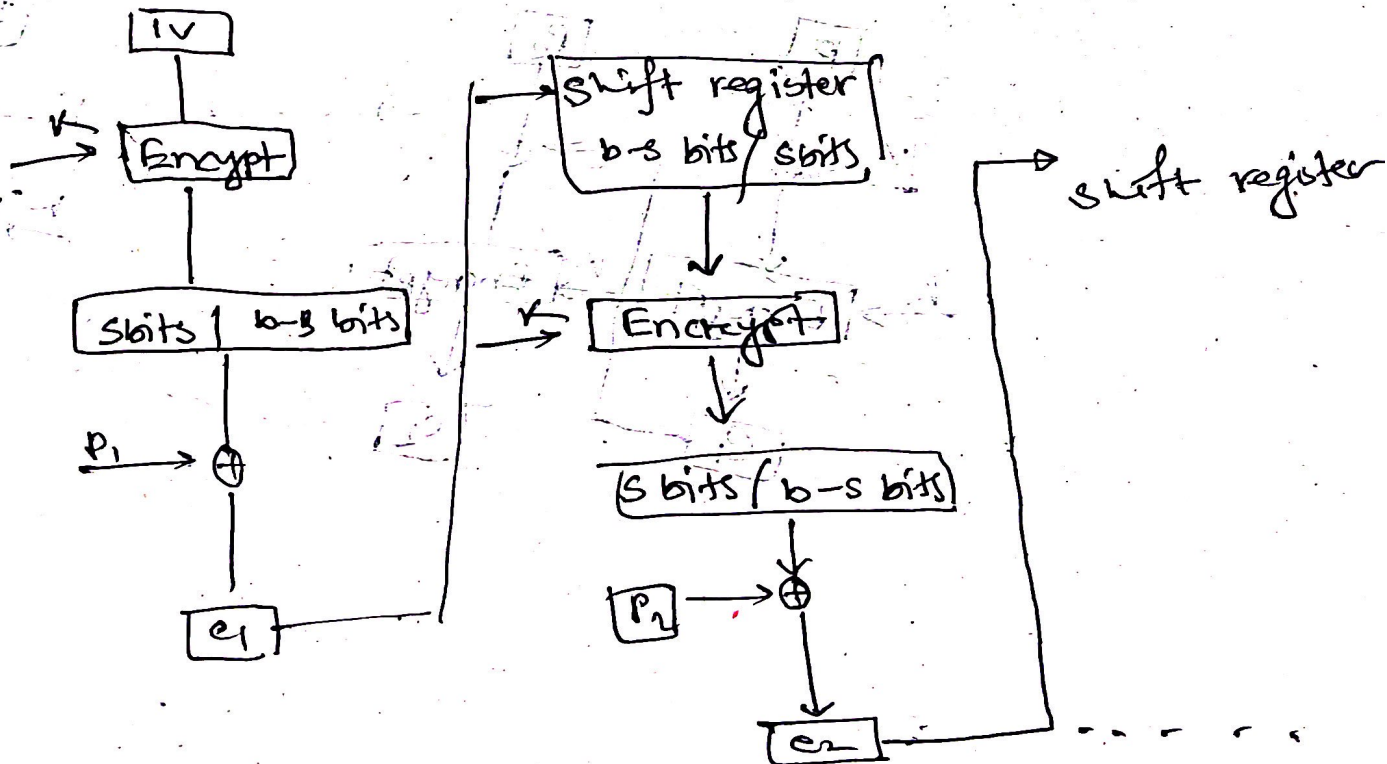


Decryption :



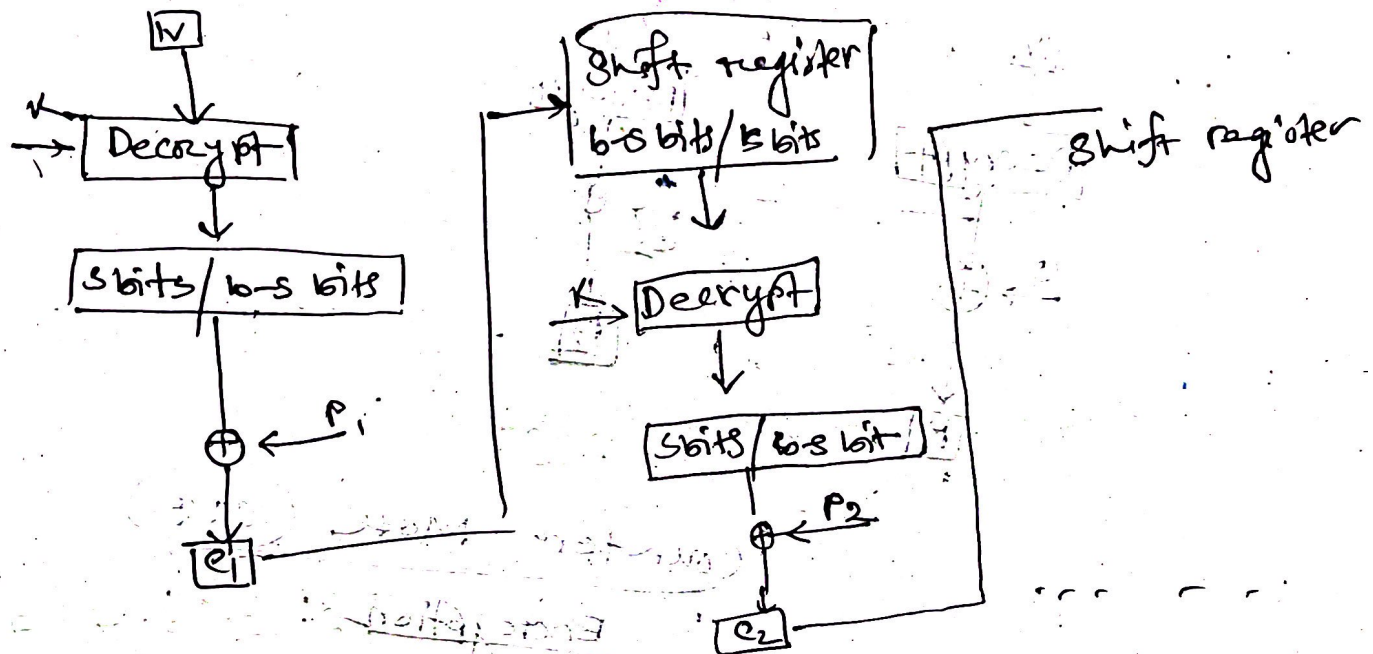
Cipher Feedback Mode (CFB)

Encryption



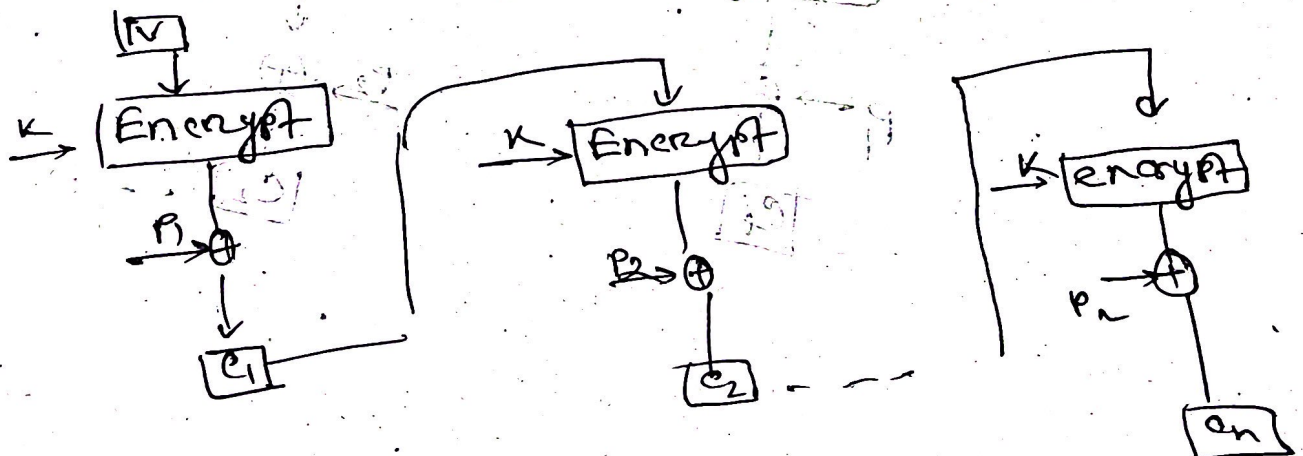
IT-21058

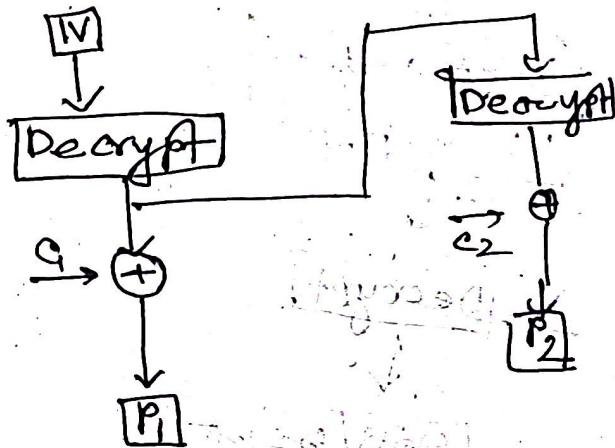
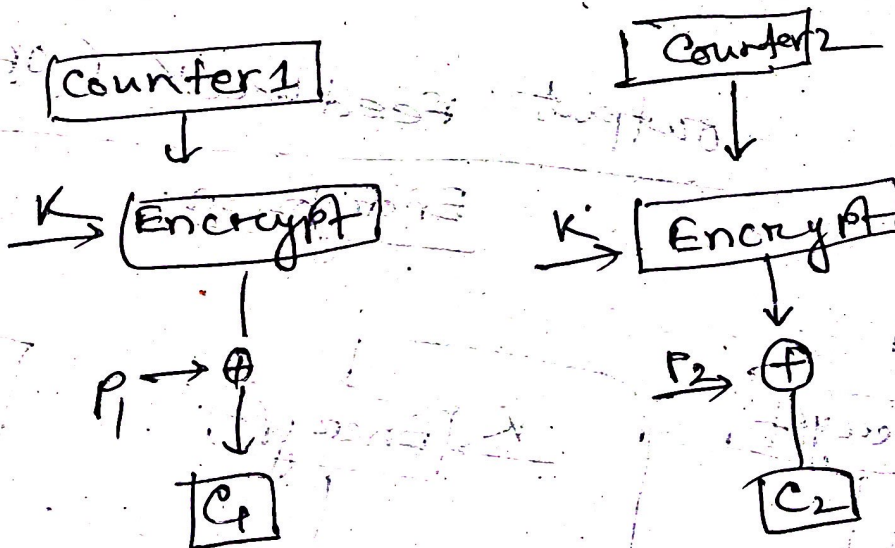
Decryption



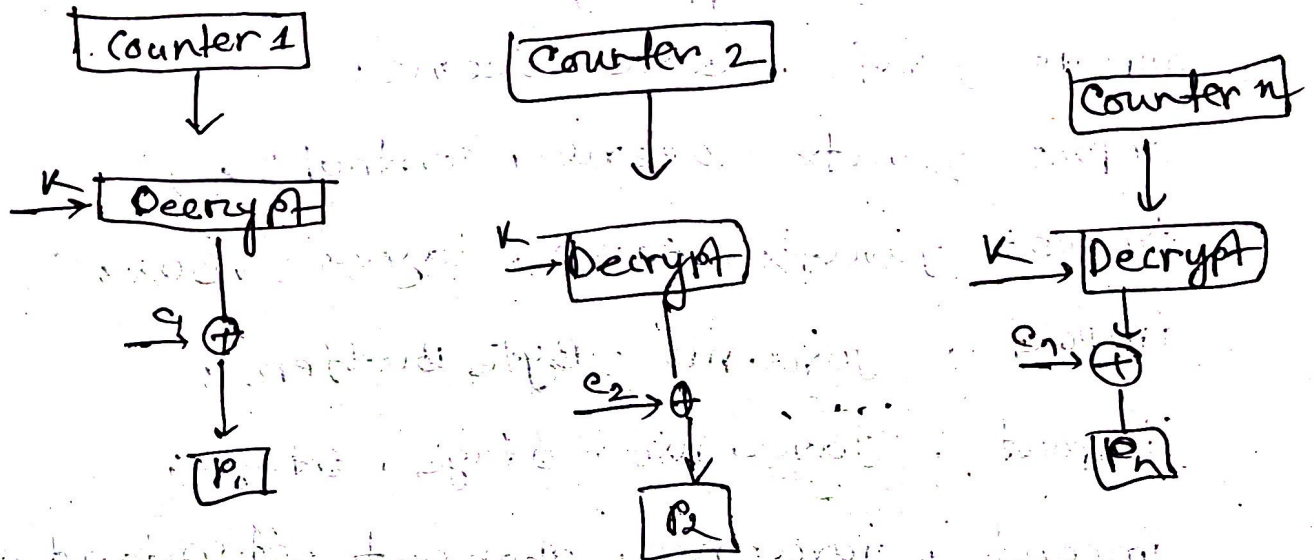
Output feedback (OFB)

Encryption

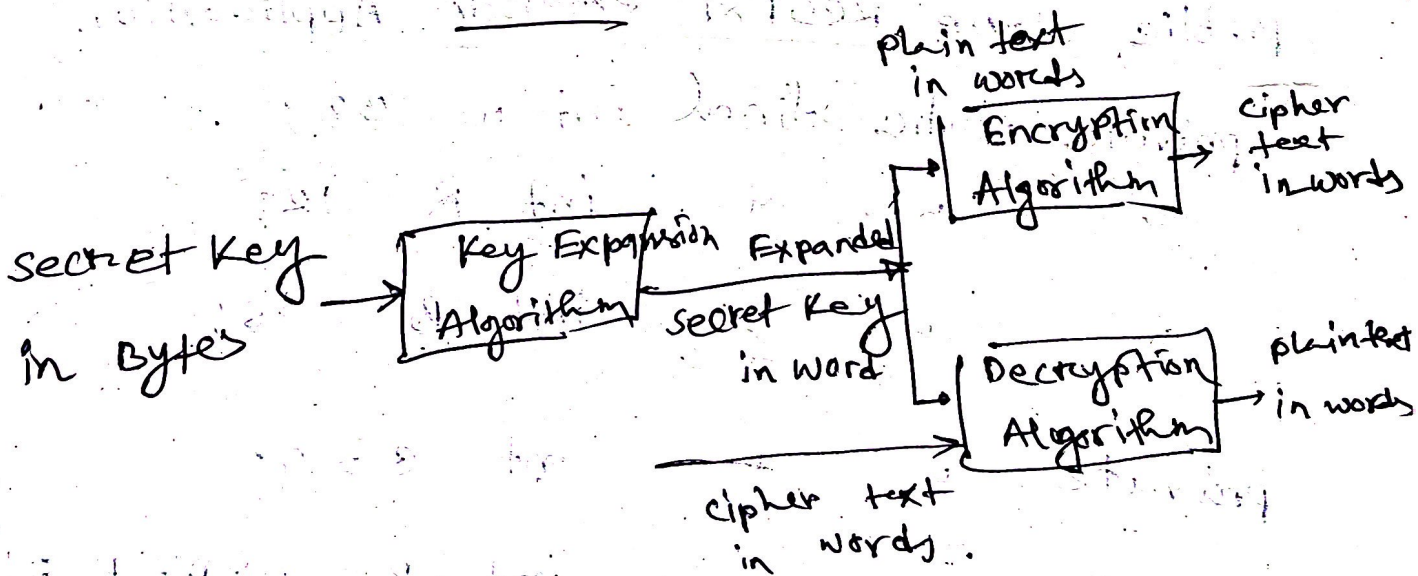


~~Feb~~DecryptionCounter Mode (CTR)Encryption

Decryption



REs



IT-2058

Code:

```
package org.example.nobfx;  
import javafx.application.Application  
import javafx.scene.scene;  
import javafx.scene.control.*;  
import javafx.scene.layout.VBox;  
import java.nio.ByteBuffer;  
import java.nio.stage.Stage;  
import java.nio.charset.StandardCharsets;  
import java.util.Array;  
  
public class R05Fx1 extends Application  
{  
    private static final int w = 32;  
    " " " " " " int R = 12;  
    " " " " " " int B = 16;  
    private " " " " int c = 4;  
    " " " " " " int p = 0xB7E15163;  
    " " " " " " int a = 0x9E3779B9;  
    " " " " " " int rot (int x, int y) {  
        " " " " " " }  
}
```

IT-21058

```
return ((x << y) | (x >>> (w - y)));
```

```
}
```

```
private static void ncs key setup (byte [] key)
```

```
{
```

```
private static void BCB key setup (byte [] key)
```

```
int [] = new int [0];
```

```
for (int i = B-1; i >= 0; i--) {
```

```
    L[i/2] = (L[i/4] << 8) + key[i] & 0xFF);
```

```
}
```

```
s[0] = P;
```

```
for (int i = 0; i < 2 * (R+1) ; i++) {
```

```
    s[i] = s[i-1] + a;
```

```
}
```

```
int A = 0, B = 0;
```

```
int i = 0, j = 0;
```

```
private static void rc5 Encrypt (int [] s, int [] data)
```

```
{
```

```
    int A = data[0]
```

```
    int B = data[i]
```


IT-21058

A = A + s[0];

B = B + s[1];

for (int i = 1; i <= R; ++i) {

A = not | (A ^ B; B) + s[2*i];

B = not | (B ^ A; A) + s[2*i+1];

}

data[0] = A;

data[1] = B;

}

private static String encryptText (String plaintext)

byte[] key = new byte[16];

int[] s = new int[2 * (R * 1)];

keySetup (key, 5);

byte[] plainBytes = plaintext.getBytes (Standard

Charsets.UTF_8);

return

```
while (buffer.hasRemaining()) {
```

```
    int A = buffer.getInt();
```

```
    int B = buffer.getInt();
```

```
    int[] data = {A, B};
```

```
    nc5.Encrypt(s, data);
```

```
    ciphertext.append(String.format("%08x
```

```
        %08x" data[0], data[1]));
```

```
}
return ciphertext.toString().trim();
}
```

@ override

```

public void start (stage . stage) {
    TextField input = new TextField ();
    input . setPromptText ("Enter English to Encrypt");
    Button encryptButton = new Button ("Encrypt");
    TextArea output = new TextArea ();
    output . setEditable (false);
    output . setWrapText (true);
    encryptButton . setOnAction (e → {
        String plaintext = input . getText ();
        if (plaintext != null && ! plaintext . isEmpty ())
            String cipherText = encrypt . Text (plaintext)
        }
    }
    else
    {
        output . setText ("Please enter some text")
    }
}

```

```
VBox.layout = new VBox (30 input encryptButton)
layout.setStyle ("-fx.padding; 20;");
```

@ override

```
public void start (Stage stage) {
    TextField input = new TextField ();
    input.setPromptText ("Enter English to encrypt");
    Button encryptButton = new Button ("Encrypt");
    TextArea output = new TextArea ();
    output.setWrapText (true);
    encryptButton.setOnAction (e -> {
        String plainText = input.getText ();
        if (plainText != null && ! plainText.isEmpty ()) {
            String cipherText = encrypt.Text (plainText);
        }
        else {
            output.setText ("Please enter some text");
        }
    });
}
```


IT-21058

```
vBox.layout = new VBox (10, input on onypButton,  
output);
```

```
layout.setStyle ("-fx.padding; 20;");
```

```
Scene scene = new Scene (layout, 500, 300);
```

```
Stage.setScene (scene);
```

```
Stage.show();
```

```
public static void main (String [] args) {  
    launch (args)
```

```
}
```

```
}
```

output

input : hello world

padding into 64 bit blocks

Block 1 : "hello " (8 bytes)

Block 2 : " rld10101010 " (Padded)

Encrypted output

07c9f2a2, 61557b56, 2493503d

original : hello world